



山东大学

信息科学与工程学院

2024 – 2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 1.2 Debug 指令的运用

专 业 班 级 通信工程 三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 2 月 23 日

实验报告

【上篇】查看 CPU 和内存，用机器指令和汇编指令编程

【实验目的】

掌握 **DEBUG** 的基本命令及其功能掌握 **win7 win8** 使用 **DEBUG** 功能

【实验具体内容】

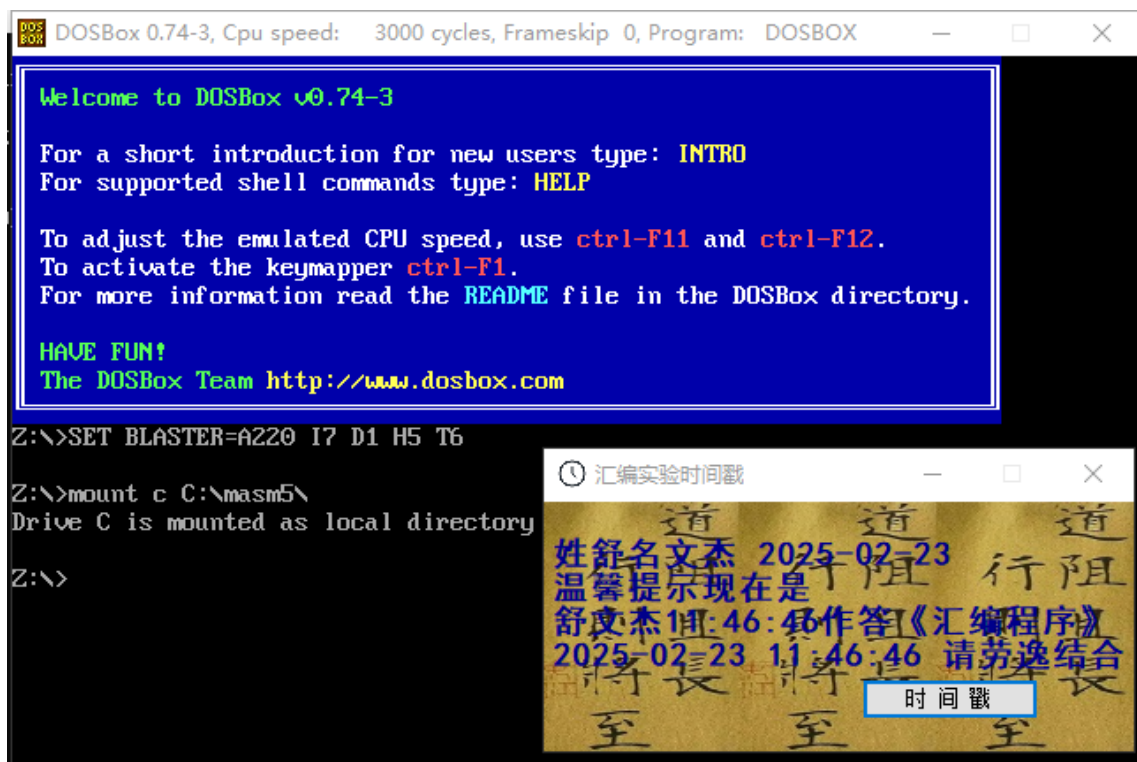
【实验 1】

(1) 使用 Debug，将下面的程序段写入内存，逐条执行，观察每条指令执行后，CPU 中相关寄存器中内容的变化。

实验源代码	
机器码	汇编指令
b8 20 4e	mov ax,4E20H
05 16 14	add ax,1416H
bb 00 20	mov bx,2000H
01 d8	add ax,bx
89 c3	mov bx,ax
01 d8	add ax,bx
b8 1a 00	mov ax,001AH
bb 26 00	mov bx,0026H
00 d8	add al,bl
00 dc	add ah,bl
00 c7	add bh,al
b4 00	mov ah,0
00 d8	add al,bl
04 9c	add al,9CH

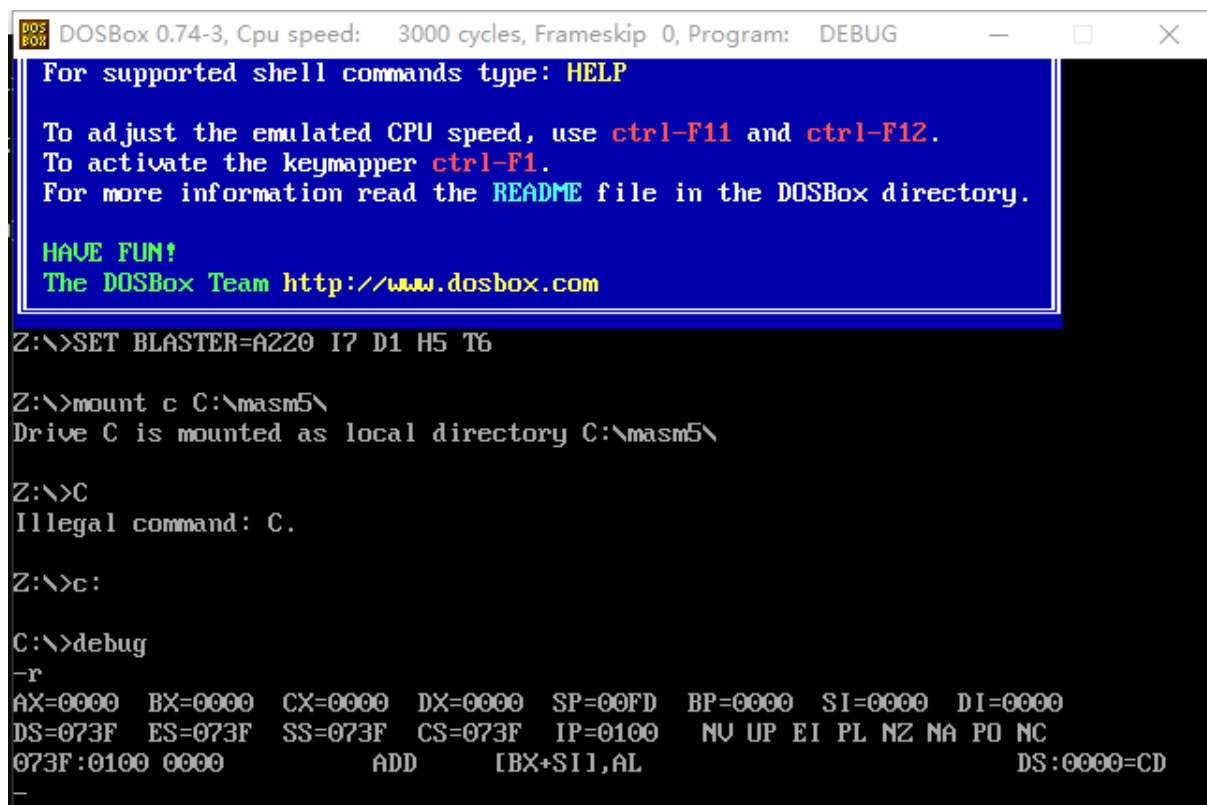
(3) 实验代码、过程、相应结果（截图）：

[1] 加载虚拟环境 C:\



图表 1

[2] 进入 debug 模式



图表 2

[3] 使用 R 指令查看当前寄存器

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
For supported shell commands type: HELP
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.
HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount c C:\masm5\
Drive C is mounted as local directory C:\masm5\

Z:\>C
Illegal command: C.

Z:\>c:

C:\>debug
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0100  NU UP EI PL NZ NA PO NC
073F:0100 0000          ADD     [BX+SI],AL          DS:0000=CD

```

图表 3

[4] 向 1000: 0000 写入机器指令

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

Z:\>C
Illegal command: C.

Z:\>c:

C:\>debug
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0100  NU UP EI PL NZ NA PO NC
073F:0100 0000          ADD     [BX+SI],AL          DS:0000=CD
-e 1000:0 b8 20 4e
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0100  NU UP EI PL NZ NA PO NC
073F:0100 0000          ADD     [BX+SI],AL          DS:0000=CD
-e
^ Error
-e b8 20 4e
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0100  NU UP EI PL NZ NA PO NC
073F:0100 0000          ADD     [BX+SI],AL          DS:0000=CD
-d 1000:0 f
1000:0000 B8 20 4E 00 00 00 00 00-00 00 00 00 00 00 00 00 . N.....

```

图表 4

这里，我运用了 D 指令来检验我的程序是否真的写入了内存

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e b8 20 4e
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0100  NU UP EI PL NZ NA PO NC
073F:0100 0000      ADD     [BX+SI],AL          DS:0000=CD
-d 1000:0 f
1000:0000 B8 20 4E 00 00 00 00 00-00 00 00 00 00 00 00 00 . N.....
-u 1000:0
1000:0000 B8204E      MOV     AX,4E20
1000:0003 0000      ADD     [BX+SI],AL
1000:0005 0000      ADD     [BX+SI],AL
1000:0007 0000      ADD     [BX+SI],AL
1000:0009 0000      ADD     [BX+SI],AL
1000:000B 0000      ADD     [BX+SI],AL
1000:000D 0000      ADD     [BX+SI],AL
1000:000F 0000      ADD     [BX+SI],AL
1000:0011 0000      ADD     [BX+SI],AL
1000:0013 0000      ADD     [BX+SI],AL
1000:0015 0000      ADD     [BX+SI],AL
1000:0017 0000      ADD     [BX+SI],AL
1000:0019 0000      ADD     [BX+SI],AL
1000:001B 0000      ADD     [BX+SI],AL
1000:001D 0000      ADD     [BX+SI],AL
1000:001F 0000      ADD     [BX+SI],AL
-t
    
```

图表 5

运用 U 指令反汇编

[5] 使用 T 命令执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-d 1000:0 f
1000:0000 B8 20 4E 00 00 00 00 00-00 00 00 00 00 00 00 00 . N.....
-u 1000:0
1000:0000 B8204E      MOV     AX,4E20
1000:0003 0000      ADD     [BX+SI],AL
1000:0005 0000      ADD     [BX+SI],AL
1000:0007 0000      ADD     [BX+SI],AL
1000:0009 0000      ADD     [BX+SI],AL
1000:000B 0000      ADD     [BX+SI],AL
1000:000D 0000      ADD     [BX+SI],AL
1000:000F 0000      ADD     [BX+SI],AL
1000:0011 0000      ADD     [BX+SI],AL
1000:0013 0000      ADD     [BX+SI],AL
1000:0015 0000      ADD     [BX+SI],AL
1000:0017 0000      ADD     [BX+SI],AL
1000:0019 0000      ADD     [BX+SI],AL
1000:001B 0000      ADD     [BX+SI],AL
1000:001D 0000      ADD     [BX+SI],AL
1000:001F 0000      ADD     [BX+SI],AL
-t
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0102  NU UP EI NG NZ NA PO NC
073F:0102 0000      ADD     [BX+SI],AL          DS:0000=CD
    
```

图表 6

可以发现，我们的寄存器没有发生变化，仔细观察发现，我们的 CS，IP 寄存器指向的

内存单元是错误的

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
DS=073F ES=073F SS=073F CS=073F IP=0102  NU UP EI NG NZ NA PO NC
073F:0102 0000 ADD [BX+SI],AL DS:0000=CD
-r ip
IP 0102
:0001
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0001  NU UP EI NG NZ NA PO NC
073F:0001 203EA700 AND [00A7],BH DS:00A7=00
-r ip
IP 0001
:0000
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0000  NU UP EI NG NZ NA PO NC
073F:0000 CD20 INT 20
-r cs
CS 073F
:1000
-t
AX=4E20 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NU UP EI NG NZ NA PO NC
1000:0003 0000 ADD [BX+SI],AL DS:0000=CD
    
```

修改CS与IP寄存器的指向

成功执行

图表 7

立即使用 R 命令来修改寄存器后执行终于成功

[6] 写入第二条指令执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
:1000
-t
AX=4E20 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NU UP EI NG NZ NA PO NC
1000:0003 0000 ADD [BX+SI],AL DS:0000=CD
-e 1000:3 05 16 14
-u
1000:0003 051614 ADD AX,1416
1000:0006 0000 ADD [BX+SI],AL
1000:0008 0000 ADD [BX+SI],AL
1000:000A 0000 ADD [BX+SI],AL
1000:000C 0000 ADD [BX+SI],AL
1000:000E 0000 ADD [BX+SI],AL
1000:0010 0000 ADD [BX+SI],AL
1000:0012 0000 ADD [BX+SI],AL
1000:0014 0000 ADD [BX+SI],AL
1000:0016 0000 ADD [BX+SI],AL
1000:0018 0000 ADD [BX+SI],AL
1000:001A 0000 ADD [BX+SI],AL
1000:001C 0000 ADD [BX+SI],AL
1000:001E 0000 ADD [BX+SI],AL
1000:0020 0000 ADD [BX+SI],AL
1000:0022 0000 ADD [BX+SI],AL
    
```

成功写入

图表 8

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:0003 0000      ADD     [BX+SI],AL          DS:0000=CD
-e 1000:3 05 16 14
-u
1000:0003 051614      ADD     AX,1416
1000:0006 0000      ADD     [BX+SI],AL
1000:0008 0000      ADD     [BX+SI],AL
1000:000A 0000      ADD     [BX+SI],AL
1000:000C 0000      ADD     [BX+SI],AL
1000:000E 0000      ADD     [BX+SI],AL
1000:0010 0000      ADD     [BX+SI],AL
1000:0012 0000      ADD     [BX+SI],AL
1000:0014 0000      ADD     [BX+SI],AL
1000:0016 0000      ADD     [BX+SI],AL
1000:0018 0000      ADD     [BX+SI],AL
1000:001A 0000      ADD     [BX+SI],AL
1000:001C 0000      ADD     [BX+SI],AL
1000:001E 0000      ADD     [BX+SI],AL
1000:0020 0000      ADD     [BX+SI],AL
1000:0022 0000      ADD     [BX+SI],AL
-t
AX=6236 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0006  NV UP EI PL NZ NA PE NC
1000:0006 0000      ADD     [BX+SI],AL          DS:0000=CD
    
```

图表 9

可以发现，IP 寄存器的也随之变化

[7] 写入第三条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:003A 0000      ADD     [BX+SI],AL
1000:003C 0000      ADD     [BX+SI],AL
1000:003E 0000      ADD     [BX+SI],AL
1000:0040 0000      ADD     [BX+SI],AL
1000:0042 0000      ADD     [BX+SI],AL
1000:0044 0000      ADD     [BX+SI],AL
-d
1000:0010 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0020 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0030 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0040 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0050 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0060 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0070 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
1000:0080 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
-r
AX=6236 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0006  NV UP EI PL NZ NA PE NC
1000:0006 BB0020      MOV     BX,2000
-t
AX=6236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0009  NV UP EI PL NZ NA PE NC
1000:0009 0000      ADD     [BX+SI],AL          DS:2000=00
    
```

图表 10

[8] 写入第四条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:001F 0000      ADD      [BX+SI],AL
-e 1000:9 01 dB
-u 1000:9
1000:0009 01DB      ADD      AX,BX
1000:000B 0000      ADD      [BX+SI],AL
1000:000D 0000      ADD      [BX+SI],AL
1000:000F 0000      ADD      [BX+SI],AL
1000:0011 0000      ADD      [BX+SI],AL
1000:0013 0000      ADD      [BX+SI],AL
1000:0015 0000      ADD      [BX+SI],AL
1000:0017 0000      ADD      [BX+SI],AL
1000:0019 0000      ADD      [BX+SI],AL
1000:001B 0000      ADD      [BX+SI],AL
1000:001D 0000      ADD      [BX+SI],AL
1000:001F 0000      ADD      [BX+SI],AL
1000:0021 0000      ADD      [BX+SI],AL
1000:0023 0000      ADD      [BX+SI],AL
1000:0025 0000      ADD      [BX+SI],AL
1000:0027 0000      ADD      [BX+SI],AL
-t
AX=8236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000B  OV UP EI NG NZ NA PE NC
1000:000B 0000      ADD      [BX+SI],AL                      DS:2000=00

```

→ 成功执行

图表 11

[9] 写入第五条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:000F 0000      ADD      [BX+SI],AL
1000:0011 0000      ADD      [BX+SI],AL
1000:0013 0000      ADD      [BX+SI],AL
1000:0015 0000      ADD      [BX+SI],AL
1000:0017 0000      ADD      [BX+SI],AL
1000:0019 0000      ADD      [BX+SI],AL
1000:001B 0000      ADD      [BX+SI],AL
1000:001D 0000      ADD      [BX+SI],AL
1000:001F 0000      ADD      [BX+SI],AL
1000:0021 0000      ADD      [BX+SI],AL
1000:0023 0000      ADD      [BX+SI],AL
1000:0025 0000      ADD      [BX+SI],AL
1000:0027 0000      ADD      [BX+SI],AL
-t
AX=8236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000B  OV UP EI NG NZ NA PE NC
1000:000B 0000      ADD      [BX+SI],AL                      DS:2000=00
-e 1000:B 89 c3
-t
AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000D  OV UP EI NG NZ NA PE NC
1000:000D 0000      ADD      [BX+SI],AL                      DS:8236=00

```

ax寄存器的值被赋给bx

图表 12

[10] 写入第六条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:001B 0000      ADD     [BX+SI],AL
1000:001D 0000      ADD     [BX+SI],AL
1000:001F 0000      ADD     [BX+SI],AL
1000:0021 0000      ADD     [BX+SI],AL
1000:0023 0000      ADD     [BX+SI],AL
1000:0025 0000      ADD     [BX+SI],AL
1000:0027 0000      ADD     [BX+SI],AL
-t
AX=8236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000B  OV UP EI NG NZ NA PE NC
1000:000B 0000      ADD     [BX+SI],AL          DS:2000=00
-e 1000:B 89 c3
-t
AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000D  OV UP EI NG NZ NA PE NC
1000:000D 0000      ADD     [BX+SI],AL          DS:8236=00
-e 1000:D 01 d8
-t
AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000F  OV UP EI PL NZ NA PE CY
1000:000F 0000      ADD     [BX+SI],AL          DS:8236=00

```

图表 13

[11] 写入第七条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:0027 0000      ADD     [BX+SI],AL
-t
AX=8236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000B  OV UP EI NG NZ NA PE NC
1000:000B 0000      ADD     [BX+SI],AL          DS:2000=00
-e 1000:B 89 c3
-t
AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000D  OV UP EI NG NZ NA PE NC
1000:000D 0000      ADD     [BX+SI],AL          DS:8236=00
-e 1000:D 01 d8
-t
AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000F  OV UP EI PL NZ NA PE CY
1000:000F 0000      ADD     [BX+SI],AL          DS:8236=00
-e 1000:F b8 1a 00
-t
AX=001A BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0012  OV UP EI PL NZ NA PE CY
1000:0012 0000      ADD     [BX+SI],AL          DS:8236=00

```

图表 14

[12] 写入第八条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:B 89 c3
-t

AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000D OV UP EI NG NZ NA PE NC
1000:000D 0000 ADD [BX+SI],AL DS:8236=00
-e 1000:D 01 dB
-t

AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000F OV UP EI PL NZ NA PE CY
1000:000F 0000 ADD [BX+SI],AL DS:8236=00
-e 1000:F b8 1a 00
-t

AX=001A BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0012 OV UP EI PL NZ NA PE CY
1000:0012 0000 ADD [BX+SI],AL DS:8236=00
-e 1000:12 bb 26 00
-t

AX=001A BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0015 OV UP EI PL NZ NA PE CY
1000:0015 0000 ADD [BX+SI],AL DS:0026=FF
    
```

成功执行

图表 15

[13] 写入第九条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:D 01 dB
-t

AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=000F OV UP EI PL NZ NA PE CY
1000:000F 0000 ADD [BX+SI],AL DS:8236=00
-e 1000:F b8 1a 00
-t

AX=001A BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0012 OV UP EI PL NZ NA PE CY
1000:0012 0000 ADD [BX+SI],AL DS:8236=00
-e 1000:12 bb 26 00
-t

AX=001A BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0015 OV UP EI PL NZ NA PE CY
1000:0015 0000 ADD [BX+SI],AL DS:0026=FF
-e 1000:15 00 dB
-t

AX=0040 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0017 NV UP EI PL NZ AC PO NC
1000:0017 0000 ADD [BX+SI],AL DS:0026=FF
    
```

成功执行

图表 16

[14] 写入第十条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:F b8 1a 00
-t

AX=001A BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0012 OV UP EI PL NZ NA PE CY
1000:0012 0000 ADD [BX+SI],AL DS:8236=00
-e 1000:12 bb 26 00
-t

AX=001A BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0015 OV UP EI PL NZ NA PE CY
1000:0015 0000 ADD [BX+SI],AL DS:0026=FF
-e 1000:15 00 d8
-t

AX=0040 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0017 NU UP EI PL NZ AC PO NC
1000:0017 0000 ADD [BX+SI],AL DS:0026=FF
-e 1000:17 00 dc
-t

AX=2640 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0019 NU UP EI PL NZ NA PO NC
1000:0019 0000 ADD [BX+SI],AL DS:0026=FF
    
```

→ bx低位加在ax高位上

图表 17

[15] 写入第十一条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:12 bb 26 00
-t

AX=001A BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0015 OV UP EI PL NZ NA PE CY
1000:0015 0000 ADD [BX+SI],AL DS:0026=FF
-e 1000:15 00 d8
-t

AX=0040 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0017 NU UP EI PL NZ AC PO NC
1000:0017 0000 ADD [BX+SI],AL DS:0026=FF
-e 1000:17 00 dc
-t

AX=2640 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0019 NU UP EI PL NZ NA PO NC
1000:0019 0000 ADD [BX+SI],AL DS:0026=FF
-e 1000:19 00 c7
-t

AX=2640 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001B NU UP EI PL NZ NA PO NC
1000:001B 0000 ADD [BX+SI],AL DS:4026=00
    
```

成功执行

图表 18

[16] 写入第十二条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:15 00 d8
-t

AX=0040 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0017  NU UP EI PL NZ AC PO NC
1000:0017 0000      ADD     [BX+SI],AL      DS:0026=FF
-e 1000:17 00 dc
-t

AX=2640 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0019  NU UP EI PL NZ NA PO NC
1000:0019 0000      ADD     [BX+SI],AL      DS:0026=FF
-e 1000:19 00 c7
-t

AX=2640 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001B  NU UP EI PL NZ NA PO NC
1000:001B 0000      ADD     [BX+SI],AL      DS:4026=00
-e 1000:1B b4 00
-t

AX=0040 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001D  NU UP EI PL NZ NA PO NC
1000:001D 0000      ADD     [BX+SI],AL      DS:4026=00
    
```

AX高位写入0

图表 19

[17] 写入第十三条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:17 00 dc
-t

AX=2640 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0019  NU UP EI PL NZ NA PO NC
1000:0019 0000      ADD     [BX+SI],AL      DS:0026=FF
-e 1000:19 00 c7
-t

AX=2640 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001B  NU UP EI PL NZ NA PO NC
1000:001B 0000      ADD     [BX+SI],AL      DS:4026=00
-e 1000:1B b4 00
-t

AX=0040 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001D  NU UP EI PL NZ NA PO NC
1000:001D 0000      ADD     [BX+SI],AL      DS:4026=00
-e 1000:1D 00 d8
-t

AX=0066 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001F  NU UP EI PL NZ NA PE NC
1000:001F 0000      ADD     [BX+SI],AL      DS:4026=00
    
```

低位相加

图表 20

[18] 写入第十四条并执行

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-e 1000:19 00 c7
-t

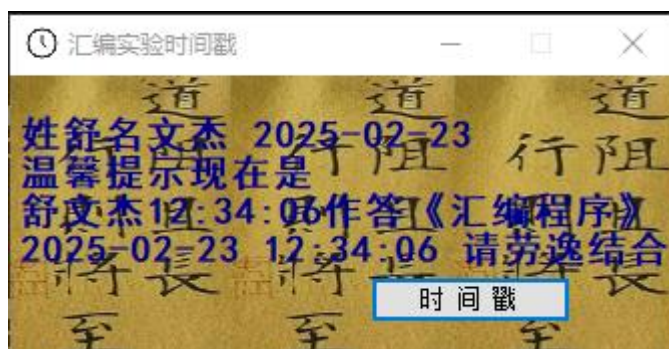
AX=2640 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001B  NV UP EI PL NZ NA PO NC
1000:001B 0000      ADD     [BX+SI],AL      DS:4026=00
-e 1000:1B b4 00
-t

AX=0040 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001D  NV UP EI PL NZ NA PO NC
1000:001D 0000      ADD     [BX+SI],AL      DS:4026=00
-e 1000:1D 00 dB
-t

AX=0066 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=001F  NV UP EI PL NZ NA PE NC
1000:001F 0000      ADD     [BX+SI],AL      DS:4026=00
-e 1000:1F 04 9c
-t
AX=0002 BX=4026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0021  NV UP EI PL NZ AC PO CY
1000:0021 0000      ADD     [BX+SI],AL      DS:4026=00
    
```

→ AX低位加上9c, 多余部分溢出

图表 21



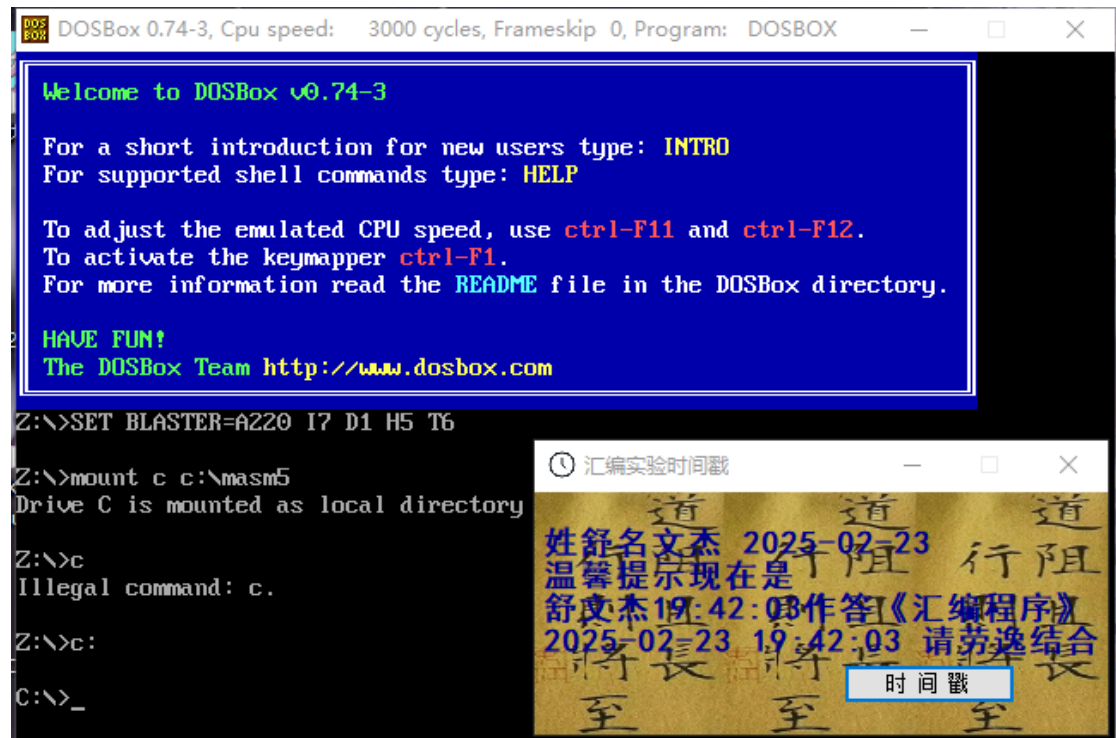
(2)将下面 3 条指令写入从 2000:0 开始的内存单元中, 利用这 3 条指令计算 2 的 8 次方。

mov ax,1 (从2000:0开始的内存单元);将1赋值给ax

add ax,ax ;ax=ax+ax (2000: 0003)

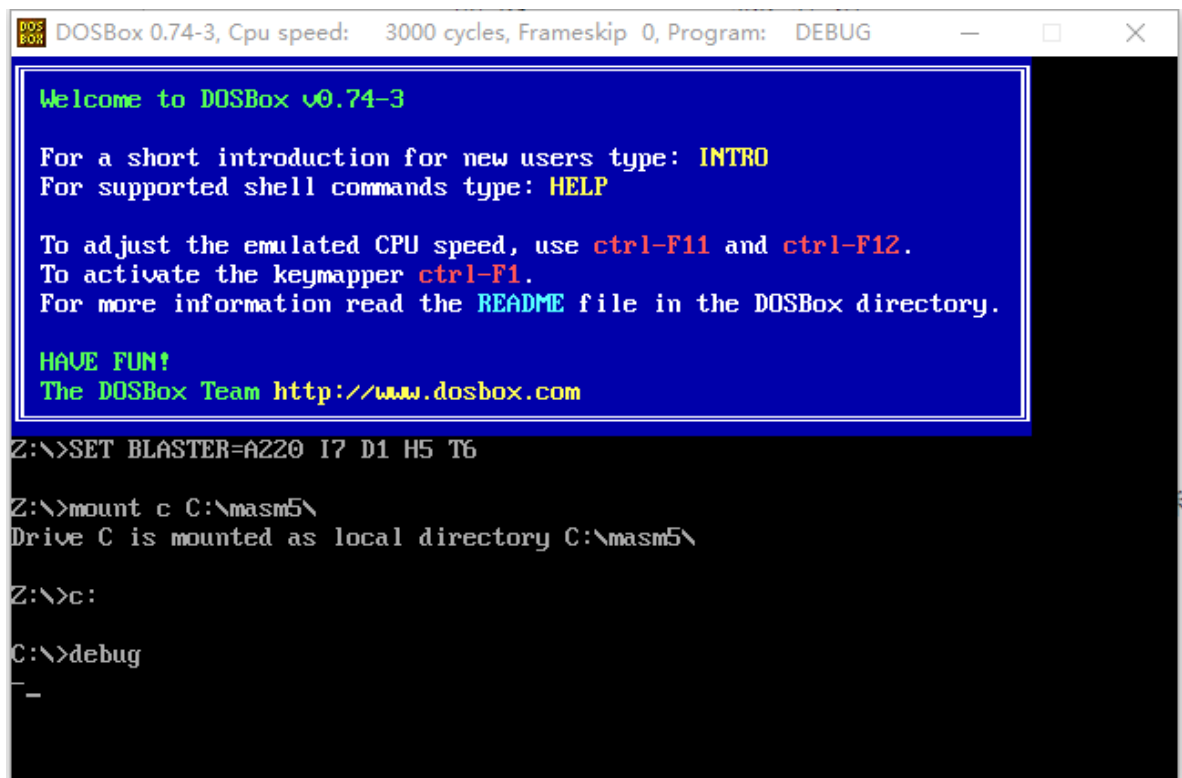
jmp 2000:0003 ;跳到上一句指令

[1] 加载虚拟环境 c:\



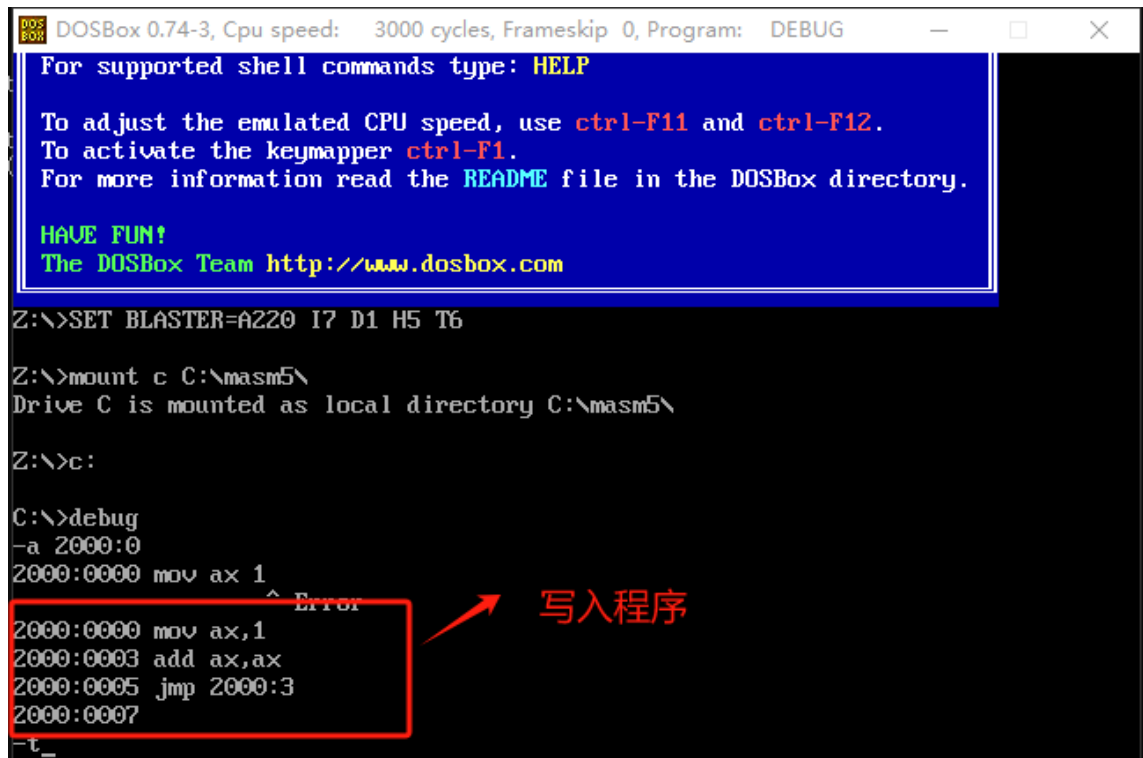
图表 22

[2] 进入 debug 模式



图表 23

[3] 使用 A 指令写入程序



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
For supported shell commands type: HELP
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.
HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount c C:\masm5\
Drive C is mounted as local directory C:\masm5\

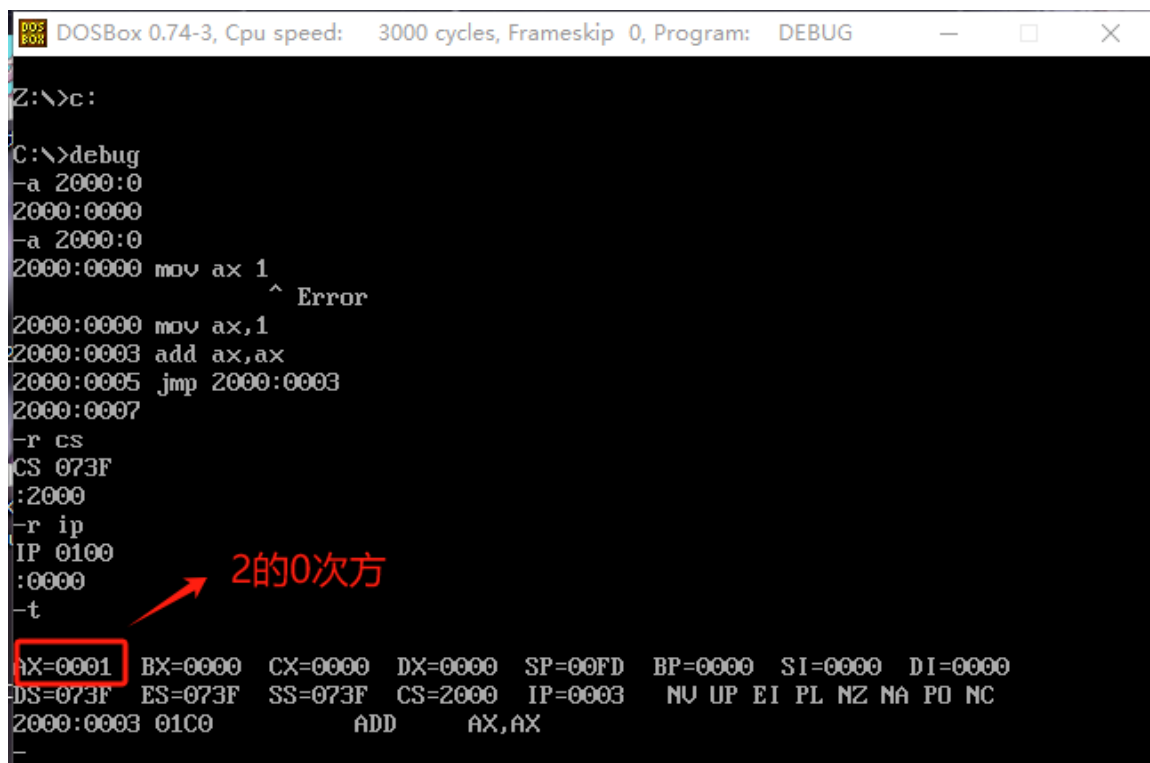
Z:\>c:

C:\>debug
-a 2000:0
2000:0000 mov ax 1
                ^ Error
2000:0000 mov ax,1
2000:0003 add ax,ax
2000:0005 jmp 2000:3
2000:0007
-t
    
```

写入程序

图表 24

[4] 重复执行



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

Z:\>c:

C:\>debug
-a 2000:0
2000:0000
-a 2000:0
2000:0000 mov ax 1
                ^ Error
2000:0000 mov ax,1
2000:0003 add ax,ax
2000:0005 jmp 2000:0003
2000:0007
-r cs
CS 073F
:2000
-r ip
IP 0100
:0000
-t
BX=0001 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0003  NU UP EI PL NZ NA PO NC
2000:0003 01C0          ADD     AX,AX
    
```

2的0次方

图表 25


```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
:2000
-r ip
IP 0100
:0000
-t
AX=0001 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0003  NU UP EI PL NZ NA PO NC
2000:0003 01C0      ADD     AX,AX
-t
AX=0002 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0005  NU UP EI PL NZ NA PO NC
2000:0005 EBFC      JMP     0003
-t
AX=0002 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0003  NU UP EI PL NZ NA PO NC
2000:0003 01C0      ADD     AX,AX
-t
AX=0004 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0005  NU UP EI PL NZ NA PO NC
2000:0005 EBFC      JMP     0003
    
```

图表 28

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=0040 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0005  NU UP EI PL NZ NA PO NC
2000:0005 EBFC      JMP     0003
-t
AX=0040 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0003  NU UP EI PL NZ NA PO NC
2000:0003 01C0      ADD     AX,AX
-t
AX=0080 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0005  NU UP EI PL NZ NA PO NC
2000:0005 EBFC      JMP     0003
-t
AX=0080 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0003  NU UP EI PL NZ NA PO NC
2000:0003 01C0      ADD     AX,AX
-t
AX=0100 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=2000 IP=0005  NU UP EI PL NZ NA PE NC
2000:0005 EBFC      JMP     0003
    
```

图表 29

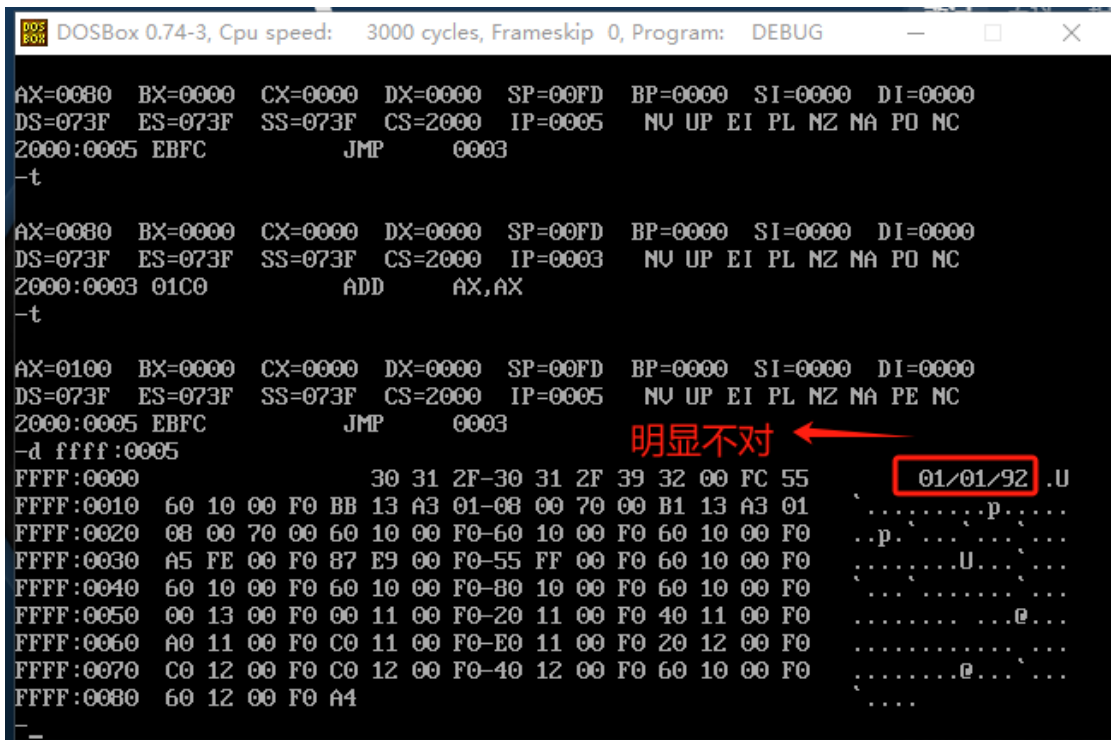
运行多次后终于算出 2 的八次方为 $16 \times 16 = 256$

(3)查看内存中的内容

PC 机主板上的 ROM 中写有一个生产日期, 在内存 FFF00H~FFFFFH 的某几个单元中, 请找到这个

生产日期并试图改变它。（内存 ffff:0005~ffff:000C(共 8 个字节单元中)处）

直接使用 D 命令查看：



图表 30

很明显，我这台电脑的主板是去年生产的，肯定不是 92 年的，于是，我上网查询了相关资料。

DOSBox 是一个虚拟环境，它并不完全模拟真实的硬件 ROM 区域。DOSBox 的内存映射没有准确地反映实际物理 ROM 的内容。

打开 cmd 指令窗，输入 wmic bios get description 与 systeminfo 指令：前者只返回了 BIOS 的版本号，只有后者才更加全面。



```

选择 命令提示符
Microsoft Windows [版本 10.0.19045.5487]
(c) Microsoft Corporation。保留所有权利。

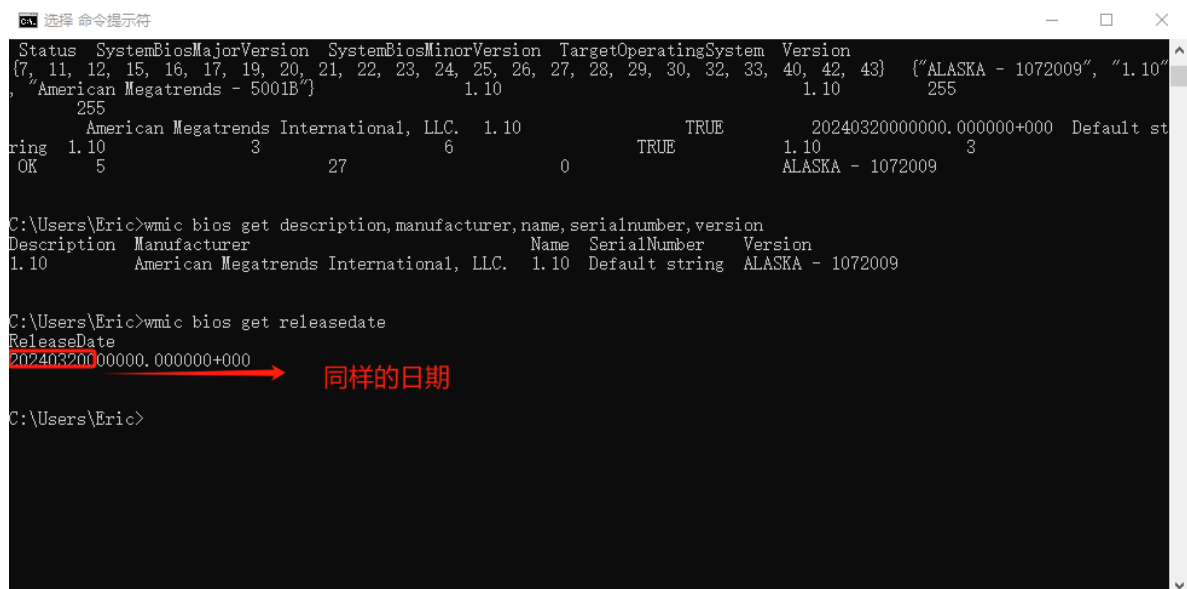
C:\Users\Eric>wmic bios get description
Description
1.10

C:\Users\Eric>systeminfo

主机名:          DESKTOP-Q7AF4G0
OS 名称:         Microsoft Windows 10 专业版
OS 版本:         10.0.19045 暂缺 Build 19045
OS 制造商:       Microsoft Corporation
OS 配置:         独立工作站
OS 构建类型:     Multiprocessor Free
注册的所有人:   Eric
注册的组织:
产品 ID:         00331-10000-00001-AA490
初始安装日期:   2024/9/4, 20:12:23
系统启动时间:   2025/2/25, 13:33:54
系统制造商:     Micro-Star International Co., Ltd.
系统型号:       MS-7E44
系统类型:       x64-based PC
处理器:          安装了 1 个处理器。
                  [01]: Intel64 Family 6 Model 151 Stepping 2 GenuineIntel ~3000 Mhz
BIOS 版本:       American Megatrends International, LLC. 1.10, 2024/3/20
Windows 目录:    C:\Windows
系统目录:        C:\Windows\system32
启动设备:        \Device\HarddiskVolume1
    
```

发现第二个命令成功执行，同时，我这块主板大概是 2024 年 3 月 20 号生产的（因为只有 BIOS 的版本日期）。

上网查询更多能够查询主板 BIOS 的生产日期的命令，终于找到了一条命令：`wmic bios get releasedate`，执行：



```

选择 命令提示符

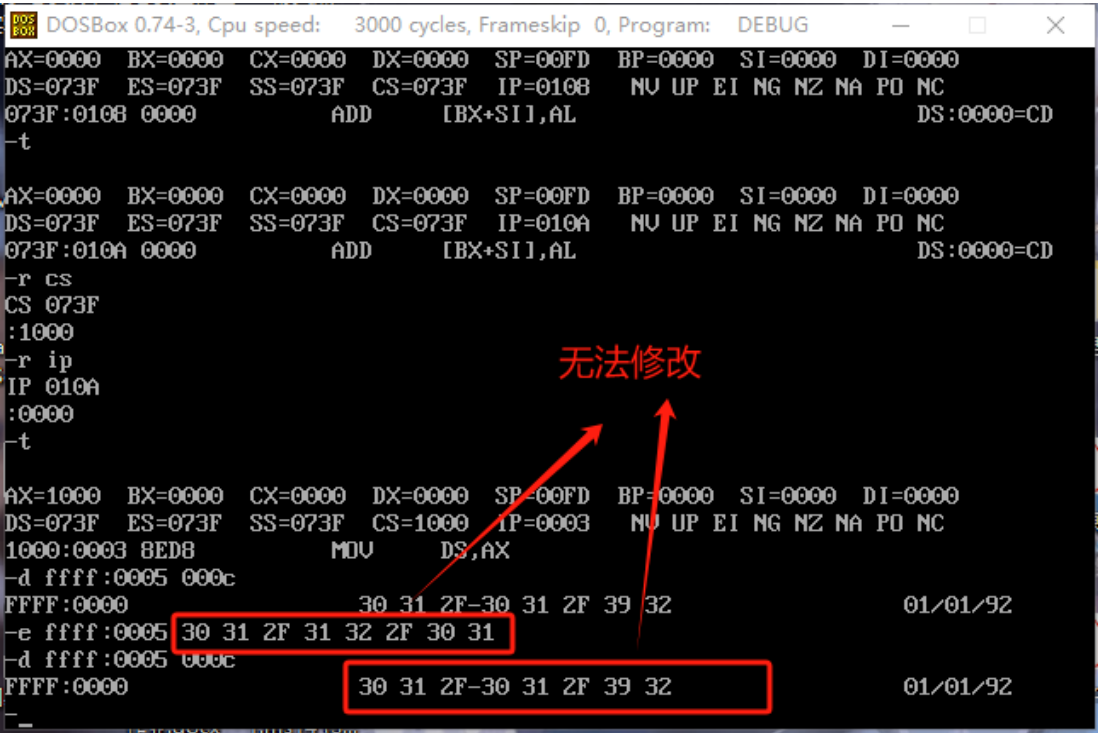
Status SystemBiosMajorVersion SystemBiosMinorVersion TargetOperatingSystem Version
{7, 11, 12, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 32, 33, 40, 42, 43} {"ALASKA - 1072009", "1.10"}
, "American Megatrends - 5001B") 1.10 1.10 255

American Megatrends International, LLC. 1.10 TRUE 2024032000000.000000+000 Default string
ring 1.10 3 6 TRUE 1.10 3
OK 5 27 0 ALASKA - 1072009

C:\Users\Eric>wmic bios get description,manufacturer,name,serialnumber,version
Description Manufacturer Name SerialNumber Version
1.10 American Megatrends International, LLC. 1.10 Default string ALASKA - 1072009

C:\Users\Eric>wmic bios get releasedate
ReleaseDate
2024032000000.000000+000
    
```

成功找到了正确的日期。



图表 31

最后，只读 ROM 是无法修改的



【第二个实验】用机器指令和汇编指令编程

(1) 使用 Debug，将下面的程序段写入内存，逐条执行，根据指令执行后的实际运行情况填空。

实验源代码
<pre>mov ax,ffff mov ds,ax mov ax,2200 mov ss,ax mov sp,0100 mov ax,[0] ;ax= C0EAH add ax,[2] ;ax= C0FCH</pre>

```

mov bx,[4] ;bx= 30F0H
add bx,[6] ;bx= 6021H
push ax ;sp= 00FE 内存单元的地址是 2200:00FE 0100
内容为: C0FCH (实际为 FC C0 因为栈是倒着存的)
push bx ;sp= 00FC 内存单元的地址是 2200:00FC 00FD
内容为 : 6021H(实际为 21 60 因为栈是倒着存的)
pop ax ;sp=00FE ;ax= 6021H
pop bx ;sp=0100 ;bx= C0FCH
push [4] ;sp=00FE ;修改的内存单元的地址是 2200:00FE 0100
内容为 30F0H(实际为 F0 30 因为栈是倒着存的)
push [6] ;sp=00FC ;修改的内存单元的地址是 2200:00FC 00FD
内容为 2F31H(实际为 31 2F 因为栈是倒着存的)
    
```

1) 利用 A 指令写入程序，并执行第一步



图表 32

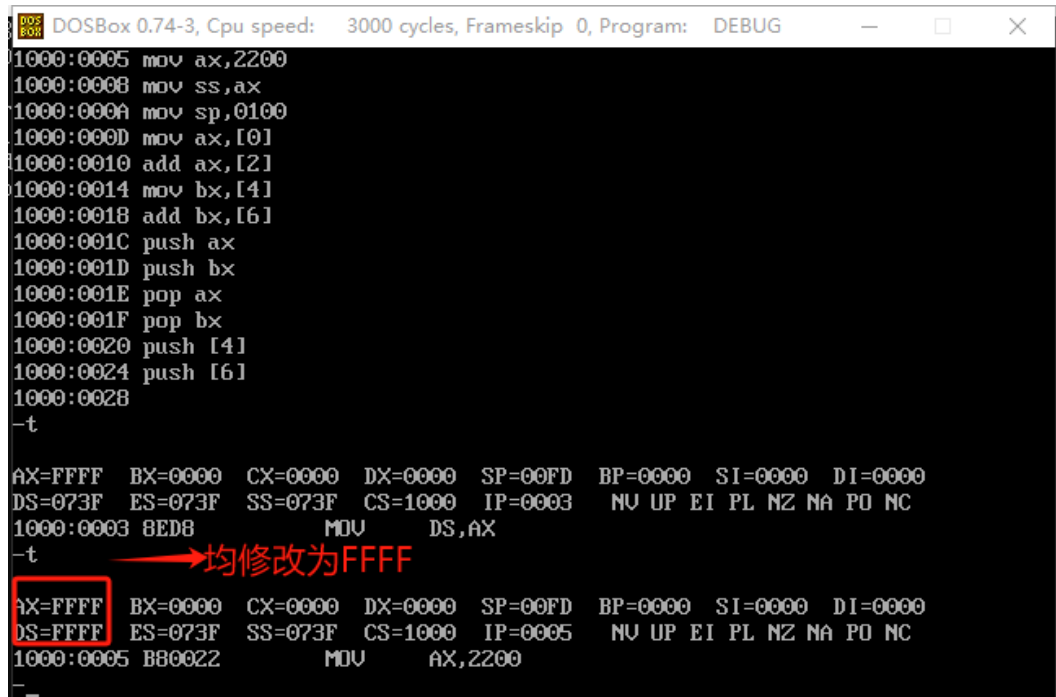
```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
-a
073F:0100
-a 1000:0000
1000:0000 mov ax,ffff
1000:0003 mov ds,ax
1000:0005 mov ax,2200
1000:0008 mov ss,ax
1000:000A mov sp,0100
1000:000D mov ax,[0]
1000:0010 add ax,[2]
1000:0014 mov bx,[4]
1000:0018 add bx,[6]
1000:001C push ax
1000:001D push bx
1000:001E pop ax
1000:001F pop bx
1000:0020 push [4]
1000:0024 push [6]
1000:0028
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F CS=1000 IP=0003  NU UP EI PL NZ NA PO NC
1000:0003 8ED8      MOV     DS,AX
-t_
    
```

图表 33

发现 FFFF 被赋值给 AX

2) 执行第二条(将 AX 里的值赋值给 DS 段寄存器)



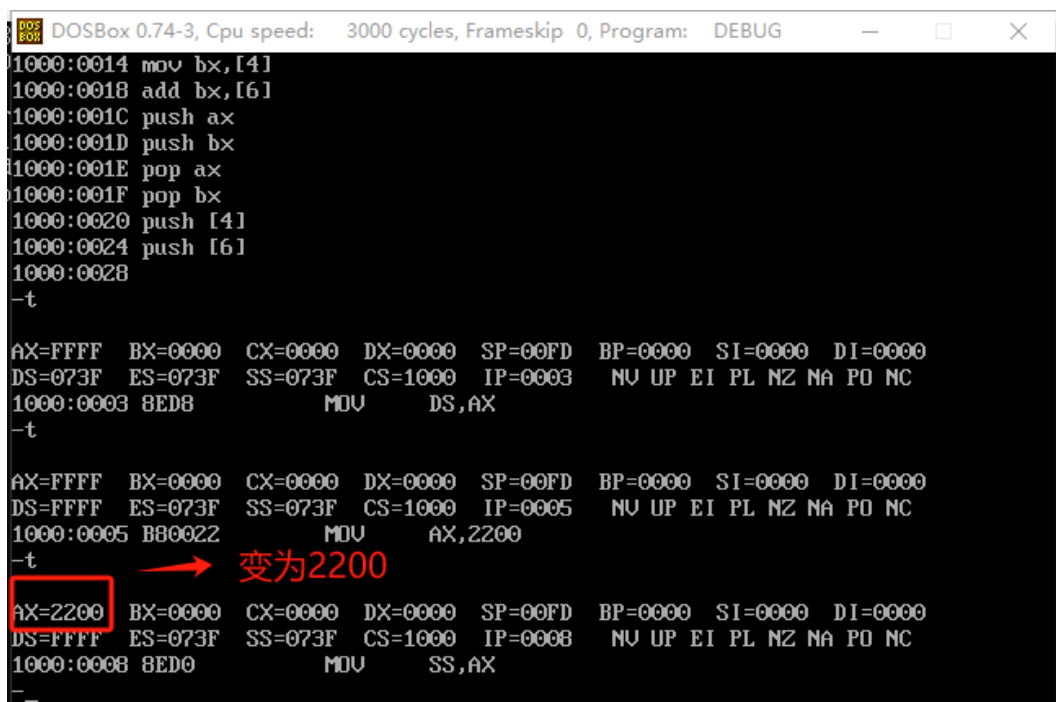
```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:0005 mov ax,2200
1000:0008 mov ss,ax
1000:000A mov sp,0100
1000:000D mov ax,[0]
1000:0010 add ax,[2]
1000:0014 mov bx,[4]
1000:0018 add bx,[6]
1000:001C push ax
1000:001D push bx
1000:001E pop ax
1000:001F pop bx
1000:0020 push [4]
1000:0024 push [6]
1000:0028
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NU UP EI PL NZ NA PO NC
1000:0003 8ED8      MOV     DS,AX
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0005  NU UP EI PL NZ NA PO NC
1000:0005 B80022    MOV     AX,2200

```

图表 34

3) 执行第三条(把 2200 赋值给 Ax)



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:0014 mov bx,[4]
1000:0018 add bx,[6]
1000:001C push ax
1000:001D push bx
1000:001E pop ax
1000:001F pop bx
1000:0020 push [4]
1000:0024 push [6]
1000:0028
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NU UP EI PL NZ NA PO NC
1000:0003 8ED8      MOV     DS,AX
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0005  NU UP EI PL NZ NA PO NC
1000:0005 B80022    MOV     AX,2200
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0008  NU UP EI PL NZ NA PO NC
1000:0008 8ED0      MOV     SS,AX

```

图表 35

4) 执行第四条(将 AX 里的值赋值给 SS 段寄存器)

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
1000:001F pop bx
1000:0020 push [4]
1000:0024 push [6]
1000:0028
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003 NU UP EI PL NZ NA PO NC
1000:0003 8ED8 MOV DS,AX
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0005 NU UP EI PL NZ NA PO NC
1000:0005 B80022 MOV AX,2200
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0008 NU UP EI PL NZ NA PO NC
1000:0008 8ED0 MOV SS,AX
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=000D NU UP EI PL NZ NA PO NC
1000:000D A10000 MOV AX,[0000] DS:0000=C0EA
    
```

两步一步执行

图表 36

观察 IP 寄存器的指向可以发现，这里两步并作了一步执行。

5) 执行第六条（把 FFFF:0000 处的数据存入 AX）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003 NU UP EI PL NZ NA PO NC
1000:0003 8ED8 MOV DS,AX
-t
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0005 NU UP EI PL NZ NA PO NC
1000:0005 B80022 MOV AX,2200
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0008 NU UP EI PL NZ NA PO NC
1000:0008 8ED0 MOV SS,AX
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=000D NU UP EI PL NZ NA PO NC
1000:000D A10000 MOV AX,[0000] DS:0000=C0EA
-t
AX=C0EA BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0010 NU UP EI PL NZ NA PO NC
1000:0010 03060200 ADD AX,[0002] DS:0002=0012
    
```

变为C0EA

图表 37

6) 执行第七条（把 FFFF:0002 处的数据加上 AX）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=FFFF BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0005 NU UP EI PL NZ NA PO NC
1000:0005 B80022 MOV AX,2200
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0008 NU UP EI PL NZ NA PO NC
1000:0008 8ED0 MOV SS,AX
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=000D NU UP EI PL NZ NA PO NC
1000:000D A10000 MOV AX,[0000] DS:0000=C0EA
-t
AX=C0EA BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0010 NU UP EI PL NZ NA PO NC
1000:0010 03060200 ADD AX,[0002] DS:0002=0012
-t
AX=C0FC BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0014 NU UP EI NG NZ NA PE NC
1000:0014 8B1E0400 MOV BX,[0004] DS:0004=30F0
-t
    
```

变为C0FC

图表 38

7) 执行第八条（把 FFFF:0004 处的数据存入 BX）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=2200 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=073F CS=1000 IP=0008 NU UP EI PL NZ NA PO NC
1000:0008 8ED0 MOV SS,AX
-t
AX=2200 BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=000D NU UP EI PL NZ NA PO NC
1000:000D A10000 MOV AX,[0000] DS:0000=C0EA
-t
AX=C0EA BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0010 NU UP EI PL NZ NA PO NC
1000:0010 03060200 ADD AX,[0002] DS:0002=0012
-t
AX=C0FC BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0014 NU UP EI NG NZ NA PE NC
1000:0014 8B1E0400 MOV BX,[0004] DS:0004=30F0
-t
AX=C0FC BX=30F0 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0018 NU UP EI NG NZ NA PE NC
1000:0018 031E0600 ADD BX,[0006] DS:0006=2F31
-t
    
```

变为30F0

图表 39

8) 执行第九条（把 FFFF:0006 处的数据加上 BX）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=2200 BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=000D NU UP EI PL NZ NA PO NC
1000:000D A10000      MOV     AX,[0000]          DS:0000=C0EA
-t

AX=C0EA BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0010 NU UP EI PL NZ NA PO NC
1000:0010 03060200    ADD     AX,[0002]          DS:0002=0012
-t

AX=C0FC BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0014 NU UP EI NG NZ NA PE NC
1000:0014 8B1E0400    MOV     BX,[0004]          DS:0004=30F0
-t

AX=C0FC BX=30F0 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0018 NU UP EI NG NZ NA PE NC
1000:0018 031E0600    ADD     BX,[0006]          DS:0006=2F31
-t

AX=C0FC BX=6021 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001C NU UP EI PL NZ NA PE NC
1000:001C 50          PUSH    AX
    
```

→ 变为6021

图表 40

9) 执行第十条（把 AX 值压入栈中）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=C0EA BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0010 NU UP EI PL NZ NA PO NC
1000:0010 03060200    ADD     AX,[0002]          DS:0002=0012
-t

AX=C0FC BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0014 NU UP EI NG NZ NA PE NC
1000:0014 8B1E0400    MOV     BX,[0004]          DS:0004=30F0
-t

AX=C0FC BX=30F0 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0018 NU UP EI NG NZ NA PE NC
1000:0018 031E0600    ADD     BX,[0006]          DS:0006=2F31
-t

AX=C0FC BX=6021 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001C NU UP EI PL NZ NA PE NC
1000:001C 50          PUSH    AX
    
```

→ 压入栈中

```

AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001D NU UP EI PL NZ NA PE NC
1000:001D 53          PUSH    BX
    
```

图表 41

10) 执行第十一条（把 BX 值压入栈中）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=C0FC BX=0000 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0014 NU UP EI NG NZ NA PE NC
1000:0014 8B1E0400 MOVBX,[0004] DS:0004=30F0
-t
AX=C0FC BX=30F0 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0018 NU UP EI NG NZ NA PE NC
1000:0018 031E0600 ADDBX,[0006] DS:0006=2F31
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001C NU UP EI PL NZ NA PE NC
1000:001C 50 PUSHAX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001D NU UP EI PL NZ NA PE NC
1000:001D 53 PUSHBX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001E NU UP EI PL NZ NA PE NC
1000:001E 58 POPAX
    
```

压栈，同时栈是倒着存取的

图表 42

11) 执行第十二条（将栈中的数据弹入 AX 中）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=C0FC BX=30F0 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0018 NU UP EI NG NZ NA PE NC
1000:0018 031E0600 ADDBX,[0006] DS:0006=2F31
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001C NU UP EI PL NZ NA PE NC
1000:001C 50 PUSHAX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001D NU UP EI PL NZ NA PE NC
1000:001D 53 PUSHBX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001E NU UP EI PL NZ NA PE NC
1000:001E 58 POPAX
    
```

出栈

图表 43

同时，我们发现，出栈的数据是最后存进去的 6021H

12) 执行第十二条（将栈中的数据弹入 BX 中）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=C0FC BX=6021 CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001C  NU UP EI PL NZ NA PE NC
1000:001C 50          PUSH  AX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001D  NU UP EI PL NZ NA PE NC
1000:001D 53          PUSH  BX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001E  NU UP EI PL NZ NA PE NC
1000:001E 5B          POP   AX
-t
AX=6021 BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001F  NU UP EI PL NZ NA PE NC
1000:001F 5B          POP   BX
-t
AX=6021 BX=C0FC CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0020  NU UP EI PL NZ NA PE NC
1000:0020 FF360400    PUSH  [0004]          DS:0004=30F0
    
```

→ 出栈

图表 44

同时，我们发现，出栈的数据是最后存进去的 6021H

13) 执行第十二条（把 FFFF:0004 处的数据压入栈中）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001D  NU UP EI PL NZ NA PE NC
1000:001D 53          PUSH  BX
-t
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001E  NU UP EI PL NZ NA PE NC
1000:001E 5B          POP   AX
-t
AX=6021 BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001F  NU UP EI PL NZ NA PE NC
1000:001F 5B          POP   BX
-t
AX=6021 BX=C0FC CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0020  NU UP EI PL NZ NA PE NC
1000:0020 FF360400    PUSH  [0004]          DS:0004=30F0
-t
AX=6021 BX=C0FC CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0024  NU UP EI PL NZ NA PE NC
1000:0024 FF360600    PUSH  [0006]          DS:0006=2F31
    
```

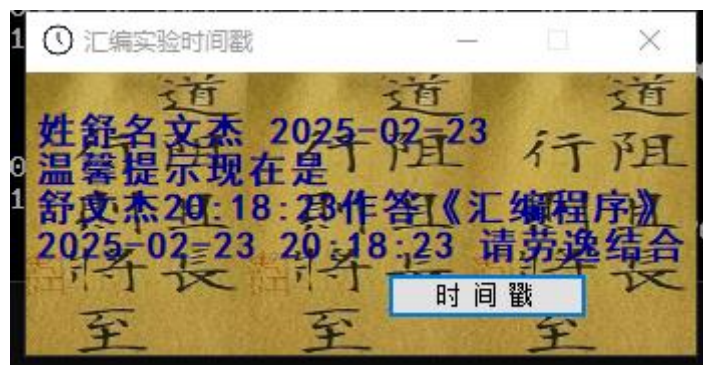
图表 45

14) 执行第十二条（把 FFFF:0006 处的数据压入栈中）

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=C0FC BX=6021 CX=0000 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001E NU UP EI PL NZ NA PE NC
1000:001E 5B POP AX
-t
AX=6021 BX=6021 CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=001F NU UP EI PL NZ NA PE NC
1000:001F 5B POP BX
-t
AX=6021 BX=C0FC CX=0000 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0020 NU UP EI PL NZ NA PE NC
1000:0020 FF360400 PUSH [0004] DS:0004=30F0
-t
AX=6021 BX=C0FC CX=0000 DX=0000 SP=00FE BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0024 NU UP EI PL NZ NA PE NC
1000:0024 FF360600 PUSH [0006] DS:0006=2F31
-t
AX=6021 BX=C0FC CX=0000 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=FFFF ES=073F SS=2200 CS=1000 IP=0028 NU UP EI PL NZ NA PE NC
1000:0028 0000 ADD [BX+SI],AL DS:C0FC=00
    
```

图表 46



图表 47

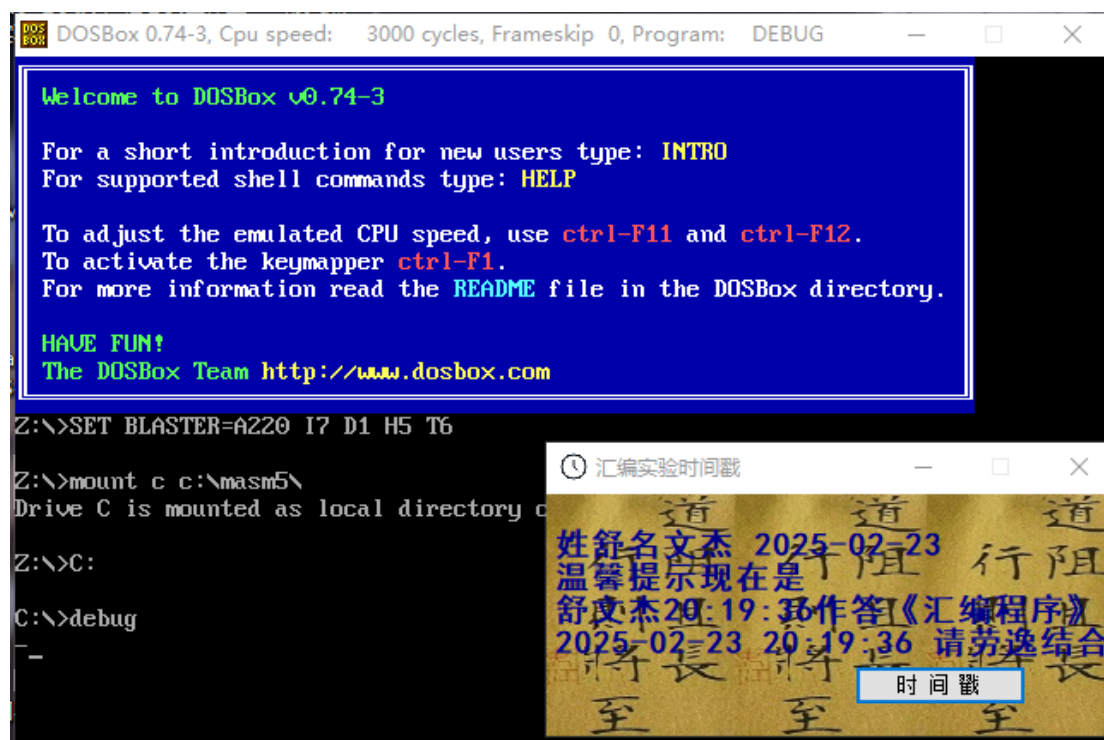
(2) 使用 Debug，将下面的程序段写入内存，逐条执行，观察每条指令执行后，CPU 中相关寄存器中内容的变化。

汇编指令

```

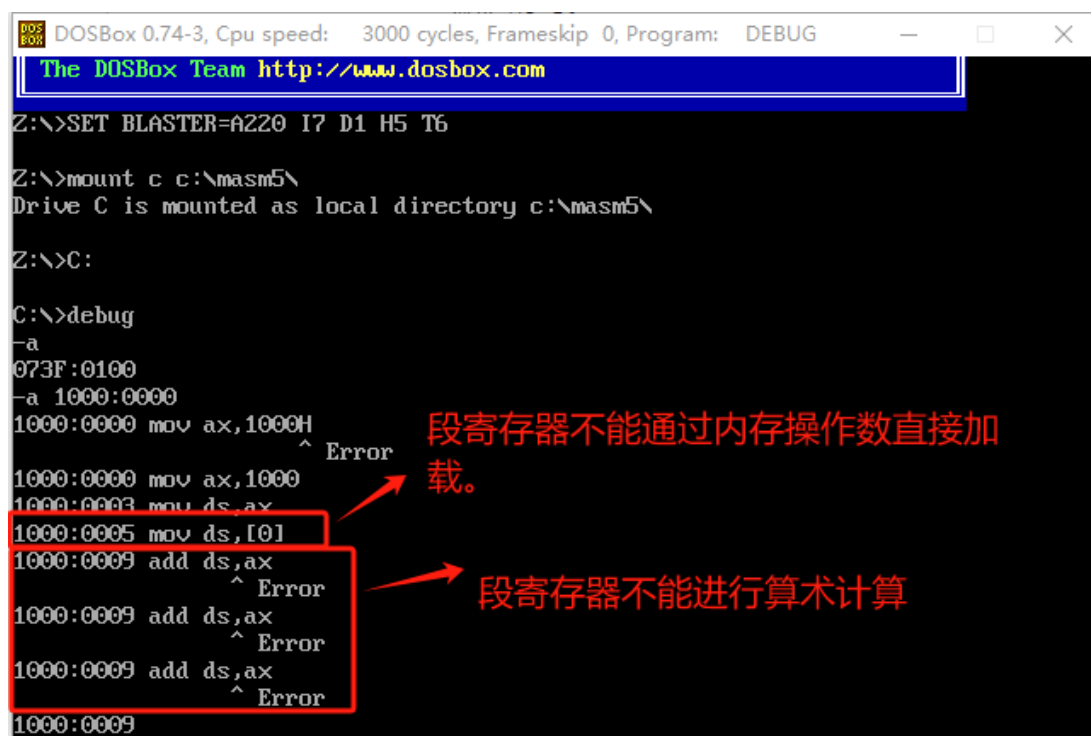
mov ax,1000H
mov ds,ax
mov ds,[0]
add ds,ax
    
```

1. 加载虚拟环境，并开启 debug 模式



图表 48

2. 利用 A 指令，写入汇编程序



图表 49

这里根据 DOSBO 编译器返回的信息和段寄存器只能通过寄存器进行中介传输，可以发现，这里有两句指令是错误的，但是为啥 1000: 0005 处的指令编译器没有报错呢？。

3. 执行第一行代码

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0106  NU UP EI NG NZ NA PO NC
073F:0106 0000      ADD     [BX+SI],AL      DS:0000=CD
-t

AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0108  NU UP EI NG NZ NA PO NC
073F:0108 0000      ADD     [BX+SI],AL      DS:0000=CD
-t

AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010A  NU UP EI NG NZ NA PO NC
073F:010A 0000      ADD     [BX+SI],AL      DS:0000=CD
-r cs
CS 073F
:1000
-r ip
IP 010A
:0000
-t
AX=1000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NU UP EI NG NZ NA PO NC
1000:0003 8ED8      MOV     DS,AX
    
```

图表 50

可以发现成功执行。

4. 执行第二行代码

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=0000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010A  NU UP EI NG NZ NA PO NC
073F:010A 0000      ADD     [BX+SI],AL      DS:0000=CD
-r cs
CS 073F
:1000
-r ip
IP 010A
:0000
-t

AX=1000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NU UP EI NG NZ NA PO NC
1000:0003 8ED8      MOV     DS,AX
-d ffff:0005 000c
FFFF:0000          30 31 2F-30 31 2F 39 32          01/01/92
-e ffff:0005 30 31 2F 31 32 2F 30 31
-d ffff:0005 000c
FFFF:0000          30 31 2F-30 31 2F 39 32          01/01/92
-t
AX=1000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=1000 ES=073F SS=073F CS=1000 IP=0005  NU UP EI NG NZ NA PO NC
1000:0005 8E1E0000  MOV     DS,[0000]      DS:0000=00B8
    
```

图表 51

5. 执行第三行代码

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
:1000
-r ip
IP 010A
:0000
-t

AX=1000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=1000 IP=0003  NV UP EI NG NZ NA PO NC
1000:0003 8ED8      MOV     DS,AX
-d ffff:0005 000c
FFFF:0000          30 31 2F-30 31 2F 39 32          01/01/92
-e ffff:0005 30 31 2F 31 32 2F 30 31
-d ffff:0005 000c
FFFF:0000          30 31 2F-30 31 2F 39 32          01/01/92
-t

AX=1000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=1000 ES=073F SS=073F CS=1000 IP=0005  NV UP EI NG NZ NA PO NC
1000:0005 8E1E0000  MOV     DS,[0000]          DS:0000=00B8
-t

AX=1000 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=00B8 ES=073F SS=073F CS=1000 IP=0009  NV UP EI NG NZ NA PO NC
1000:0009 0000      ADD     [BX+SI],AL          DS:0000=00
    
```

图表 52

令人惊讶的是，这句代码居然成功执行了，我先让 AI 为我解答一下。

在 x86 汇编语言中，`mov` 指令用于数据传输，但它的操作数有严格的限制。特别是段寄存器（如 `DS`，`ES`，`SS`，`CS`，`FS`，`GS`）的操作有一些特殊的规则。

问题：`mov` 段寄存器，内存单元 是否可以？

答案是：不可以直接这样写！

原因：

1. 段寄存器的特殊性：
 - 段寄存器是用来定义内存段的基址的特殊寄存器。
 - 它们不能像通用寄存器（如 `AX`，`BX` 等）那样自由地参与复杂的寻址或数据操作。
2. 指令格式限制：
 - 在 x86 架构中，`mov` 指令不允许直接将内存中的数据加载到段寄存器中。例如，以下代码是非法的：

```

Asm
mov ds, [some_memory_address]
    
```

图表 53

它告诉我这样是非法的，但 `debug` 模式下却又是可以执行的，我进一步从书本上和网上博客上寻找答案。

王爽《汇编语言》p53

图 3.5 试验 mov [0],cs

第 3 章 寄存器(内存访问) 53

图 3.5 中,当 CS:IP 指向 0B39:0105 的时候,Debug 显示当前的指令 mov [0000],cs,因为这是一条访问内存的指令,Debug 还显示出指令要访问的内存单元中的内容。由于指令中的 CS 是一个 16 位寄存器,所以要访问(写入)的内存单元是一个字单元,它的偏移地址为 0,段地址在 ds 中,Debug 在屏幕右边显示出“DS:0000=0000”,我们可以知道这个字单元中的内容为 0。

mov [0000],cs 执行后,CS 中的数据(0B39H)被写入 1000:0 处,1000:1 单元存放 0BH,1000:0 单元存放 39H。

最后,用 D 命令从 1000:0 开始查看指令执行后内存中的情况,注意 1000:0、1000:1 两个单元的内容。

(3) “mov 段寄存器,内存单元”也应该可行,比如我们可以用 10000H 处存放的字型数据设置 ds(即将 10000H 处存放的字型数据送入 ds),指令如下。

```

mov ax,1000H
mov ds,ax
mov ds,[0]

```

可以自行在 Debug 中进行试验。

add 和 sub 指令同 mov 一样,都有两个操作对象。它们也可以有以下几种形式。

add 寄存器,数据	比如: add ax,8
add 寄存器,寄存器	比如: add ax,bx
add 寄存器,内存单元	比如: add ax,[0]
add 内存单元,寄存器	比如: add [0],ax
sub 寄存器,数据	比如: sub ax,9
sub 寄存器,寄存器	比如: sub ax,bx
sub 寄存器,内存单元	比如: sub ax,[0]
sub 内存单元,寄存器	比如: sub [0],ax

它们可以对段寄存器进行操作吗?比如“add ds,ax”。请自行在 Debug 中试验。

3.5 数 据 段

图表 54

首先我们将DS修改回之前的值: 1000H, 然后使用A命令向内存中写入下面一条汇编指令:

mov [0],ds

修改CS:IP, 使用T命令执行, 执行后使用D命令查看内存单元内容, 观察结果:

```

-r ds
DS 073F
:1000
-a 2000:4
2000:0004 mov [0],ds
2000:0008
-r
AX=2000 BX=1000 CX=0000 DX=0000 SP=00EB BP=0000 SI=0000 DI=0000
DS=1000 ES=073F SS=073F CS=2000 IP=0004  NU UP DI PL NZ NA PO NC
2000:0004 8C1E0000      MOV     [0000],DS      DS:0000=073F
-t
AX=2000 BX=1000 CX=0000 DX=0000 SP=00EB BP=0000 SI=0000 DI=0000
DS=1000 ES=073F SS=073F CS=2000 IP=0008  NU UP DI PL NZ NA PO NC
2000:0008 8CCB      MOV     AX,CS
-d 1000:0 f
1000:0000  00 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

CSDN @奋进的代码

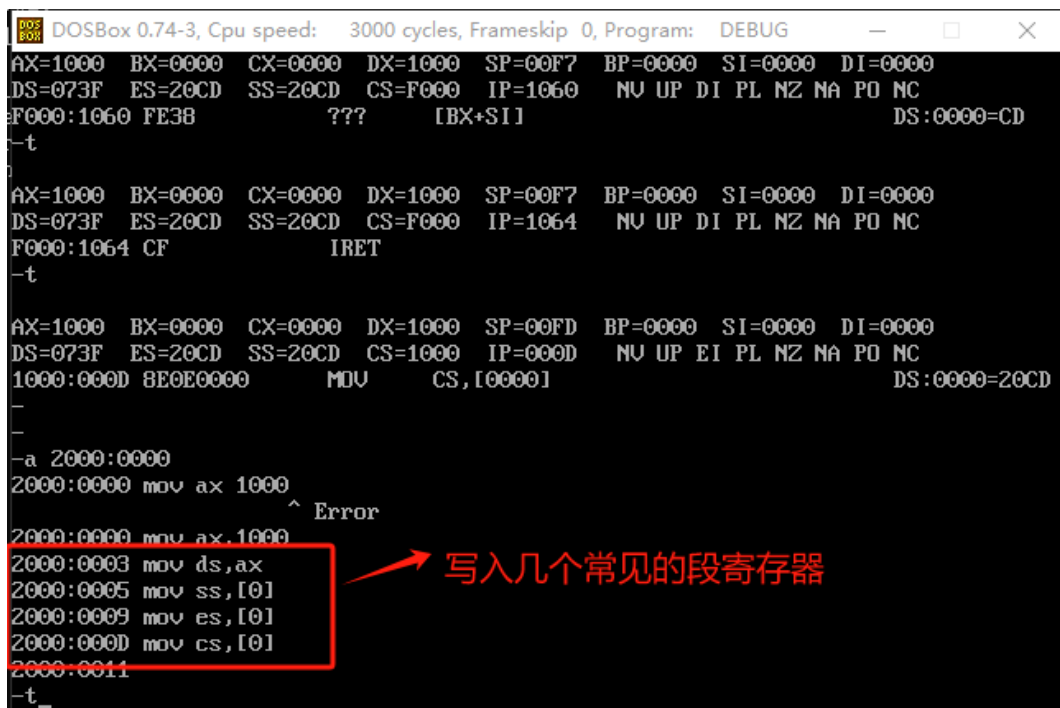
发现1000H:0字单元确实被修改为1000H, 说明 **MOV 内存单元,段寄存器** 也是可行的!

图表 55

发现二者均证明了上者的可行性，然后，我开始思考为啥 AI 会认为他是错的呢？

难道这其中有例外的段寄存器不能这样执行吗？

于是，我开始尝试：



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=073F ES=20CD SS=20CD CS=F000 IP=1060 NV UP DI PL NZ NA PO NC
F000:1060 FE3B ??? [BX+SI] DS:0000=CD
-t
AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=073F ES=20CD SS=20CD CS=F000 IP=1064 NV UP DI PL NZ NA PO NC
F000:1064 CF IRET
-t
AX=1000 BX=0000 CX=0000 DX=1000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=20CD SS=20CD CS=1000 IP=000D NV UP EI PL NZ NA PO NC
1000:000D 8E0E0000 MOV CS,[0000] DS:0000=20CD
-
-a 2000:0000
2000:0000 mov ax, 1000
2000:0000 mnu ax, 1000
2000:0003 mov ds, ax
2000:0005 mov ss, [0]
2000:0009 mov es, [0]
2000:000D mov cs, [0]
2000:0011
-t_
    
```

写入几个常见的段寄存器

图表 56



```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=073F ES=20CD SS=20CD CS=F000 IP=1064 NV UP DI PL NZ NA PO NC
F000:1064 CF IRET
-r cs
CS F000
:2000
-r ip
IP 1064
:0000
-t
AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=073F ES=20CD SS=20CD CS=2000 IP=0003 NV UP DI PL NZ NA PO NC
2000:0003 8ED8 MOV DS,AX
-t
AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=1000 ES=20CD SS=20CD CS=2000 IP=0005 NV UP DI PL NZ NA PO NC
2000:0005 8E160000 MOV SS,[0000] DS:0000=00B8
-t
AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=1000 ES=00B8 SS=00B8 CS=2000 IP=000D NV UP DI PL NZ NA PO NC
2000:000D 8E0E0000 MOV CS,[0000] DS:0000=00B8
-t_
    
```

开始执行

图表 57

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
:2000
-r ip
IP 1064
:0000
-t

AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=073F ES=20CD SS=20CD CS=2000 IP=0003  NU UP DI PL NZ NA PO NC
2000:0003 8ED8      MOV     DS,AX
-t

AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=1000 ES=20CD SS=20CD CS=2000 IP=0005  NU UP DI PL NZ NA PO NC
2000:0005 8E160000  MOV     SS,[0000]      DS:0000=00B8
-t

AX=1000 BX=0000 CX=0000 DX=1000 SP=00F7 BP=0000 SI=0000 DI=0000
DS=1000 ES=00B8 SS=00B8 CS=2000 IP=000D  NU UP DI PL NZ NA PO NC
2000:000D 8E0E0000  MOV     CS,[0000]      DS:0000=00B8
-t

AX=1000 BX=0000 CX=0000 DX=1000 SP=00F1 BP=0000 SI=0000 DI=0000
DS=1000 ES=00B8 SS=00B8 CS=F000 IP=1060  NU UP DI PL NZ NA PO NC
F000:1060 FE3B      ???     LBX+SI      DS:0000=B8
    
```

图表 58

前面几个寄存器都很好的执行了，但是 CS 寄存器并没有执行，反而跳到了主板 ROM 处的一个地址。

上网查询过后，网上有人说是开机时 CS:IP 会跳到 0XF000: 0FFF0 与当前不符，还有人说是进入了保护模式，我并没有找到一个可以让我信服的说法，但是，可以明确的是，CS 是不能用 mov 指令来修改的。

因此，AI 的说法和课本的说法都有点问题，应该排除 CS 这类特殊的段寄存器才是完全正确的。

(3) 仔细观察下图中的实验过程，然后分析：为什么 2000:0~2000:f 中的内容会发生改变？

```

C:\>debug
-a
0B39:0100 mov ax,2000
0B39:0103 mov ss,ax
0B39:0105 mov sp,10
0B39:0108 mov ax,3123
0B39:010B push ax
0B39:010C mov ax,3366
0B39:010F push ax
0B39:0110
-e 2000:0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
-d 2000:0 f
2000:0000 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B39 ES=0B39 SS=0B39 CS=0B39 IP=0100  NU UP EI PL NZ NA PO NC
0B39:0100 B80020      MOV     AX,2000
-t
AX=2000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B39 ES=0B39 SS=0B39 CS=0B39 IP=0103  NU UP EI PL NZ NA PO NC
0B39:0103 8ED0      MOV     SS,AX
-t
AX=2000 BX=0000 CX=0000 DX=0000 SP=0010 BP=0000 SI=0000 DI=0000
DS=0B39 ES=0B39 SS=2000 CS=0B39 IP=0108  NU UP EI PL NZ NA PO NC
0B39:0108 B82331      MOV     AX,3123
-d 2000:0 f
2000:0000 00 00 00 00 00 00 00 20-00 00 08 01 39 0B 9D 05 .....
    
```

这应该是在初始化栈段相关寄存器时对内存的一个标记，但是仔细观察相应的存入的数据，我们会发现这些数据并非乱码，而是寄存器的值

```

C:\>debug
-a
0B39:0100 mov ax,2000
0B39:0103 mov ss,ax
0B39:0105 mov sp,10
0B39:0108 mov ax,3123
0B39:010B push ax
0B39:010C mov ax,3366
0B39:010F push ax
0B39:0110
-e 2000:0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
-d 2000:0 f
2000:0000 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B39 ES=0B39 SS=0B39 CS=0B39 IP=0100  NU UP EI PL NZ NA PO NC
0B39:0100 B80020      MOV     AX,2000
-t
AX=2000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B39 ES=0B39 SS=0B39 CS=0B39 IP=0103  NU UP EI PL NZ NA PO NC
0B39:0103 8ED0      MOV     SS,AX
-t
AX=2000 BX=0000 CX=0000 DX=0000 SP=0010 BP=0000 SI=0000 DI=0000
DS=0B39 ES=0B39 SS=2000 CS=0B39 IP=0108  NU UP EI PL NZ NA PO NC
0B39:0108 B82331      MOV     AX,3123
-d 2000:0 f
2000:0000 00 00 00 00 00 00 00 20-00 00 08 01 39 0B 9D 05 .....
    
```

图表 59

这里找到了四个寄存器的值（最后一个没配对），但是它为什么会将这些值存在这里呢，联想到我们都是单步执行程序，并非连续，我因此怀疑是中断程序时为了让程序知道下一步指向什么，所以进行了存储。

上网查询中断相关知识，我找到了

3、8086中断响应

1、外部可屏蔽中断响应

1、等待当前指令结束，然后进入中断响应周期。

2、CPU获取中断类型号

3、当前的PSW、CS、IP的内容依次压入栈中。（保存了断点的状态和断点地址，以便返回时恢复）

4、清除PSW中的IF位和TF位。（IF=0意味着关中断，这里不包含中断嵌套）

5、把中断服务函数的入口地址置入IP和CS

6、完成响应，进入中断服务函数

看来我猜想得不错，就是外部中断响应导致的内存变化，同时我也对应出了我没有找到的那个内存对应的东西，即 **FLAGS** 寄存器，作为标志位表示程序状态。

【实验心得】

1. 汇编指令与机器码编程感受

机器码最开始给我带来的是一种繁琐厌烦的感觉，每一个语句，每一条指令，我写错了任何一位数据都会导致最后程序执行的失败，它相较汇编语言更是抛弃了对于逻辑抽象的简洁，同时，相较于汇编语言这一门编译型语言，你写的程序完全会被编译器重新检查，一些非法操作完全会被提醒，但是机器码没有这一过程，这让原本依赖编译器程序的报错来编程的我感到不适，所有的程序是否成功只有看寄存器是否成功改变，感觉回到了数电实验那样的原始阶段（只看高低电平），每一条指令在我按下 enter 键前都被检查了 3 次以上。但第一次执行时却还是没有反应，我检查了数次，最后发现原来是 CS:IP 指向的代码语句的地址错了。但是随着实验进行的深入，我发现汇编语言有着其独特的魅力，计算 2 的 8 次方，放在 C 语言里需要定义多少变量，甚至循环结构，但是汇编语言中只用短短三行结束，从头到尾只有寥寥几个寄存器，这无疑显示了它的强大。

2. 栈

栈这个数据结构既让我感到熟悉，又让我觉得陌生，熟悉说的是在高级语言中我可没少和栈打交道在函数的形参的调用，递归的执行常常有他的影子，而陌生指的是我是第一次如此清晰地看到栈相关的寄存器，以及其到底是如何工作的，SS 是如何指向内存地址，SP 指向栈顶是如何自减，它仿佛在我眼前活了过来。同时，我之前使用单片机时常常使用中断，但是我对中断是如何进行的却是一无所知。在实验任务三中，中断处理的“时间胶囊”：——栈，向我揭开了它的面纱，硬件中断发生时，CPU 自动将 FLAGS、CS、IP 压栈，处理完再

IRET 弹出恢复现场。这种机制让我惊叹于栈的“时空冻结”能力——它默默保存了程序被中断前的完整状态。我得以窥见其一角。

3. mov 指令的理解与探究

在做实验下篇的第二个实验任务,我先根据我浅薄的汇编知识找到了两个错误,其一是 mov 指令不能进行段寄存器,内存单元的命令与 add 指令不能直接操作段寄存器,在后续的实验中,我发觉我只对了一半,后半是对的,而前半是错误的,在疑惑下,我先去问了千问,给出了与我一样的答案,但是这明明可以执行,我进一步翻书和查找网上的论坛,发现 mov 指令进行段寄存器,内存单元是合法的,但是我最后的实验又否定了这一点,CS 寄存器只能通过 jmp 来改变,这个过程可谓曲折了。书本与 AI 都是错误的,要不是我最终进行了实践,发现真理的机会就要与我失之交臂了,因此,我觉得我们应该始终保持一种质疑的精神,尽信书不如不看书,自己去探索。

同时我也将这个问题问了多个 AI 助手:

Deepseek:



它也不能正确回答问题。
豆包：

8086中mov 段寄存器 内存单元 是合法的吗

在 8086 汇编语言中，`mov 段寄存器, 内存单元` 这种指令形式**不合法**。下面从 8086 的指令规则以及原因来详细解释：

规则说明

在 8086 汇编里，不允许直接将内存单元的内容传送到段寄存器（如 CS、DS、ES、SS）。这是因为 8086 的硬件设计和指令系统对段寄存器的操作有一定的限制。

错误示例

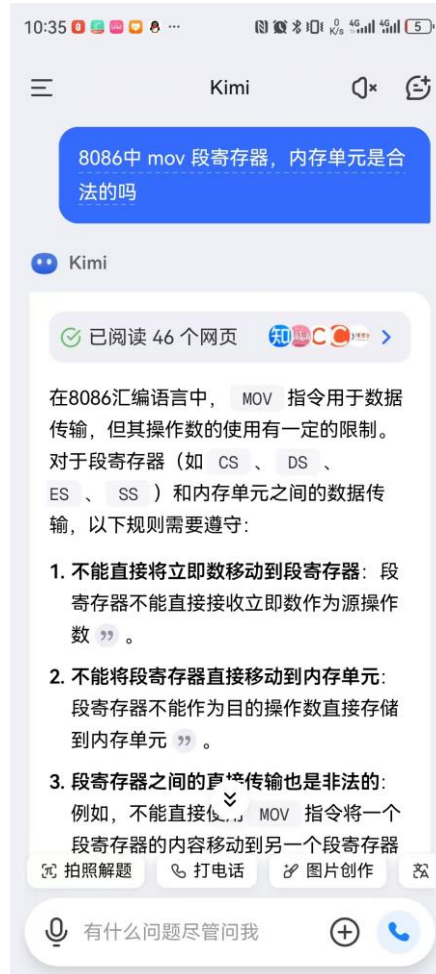
下面的代码是不合法的：

收起 ^

```
asm
; 假设DATA是一个内存单元的地址
DATA DB 10H
MOV DS, DATA ; 这是错误的指令
```

它也不能正确回答问题。

Kimi:



Kimi 同上

最后杀死比赛的居然是 Grok3:

9:51

您处于离线状态

8086汇编中 MOV 段寄存器 内存单元是合法的吗?

在 8086 汇编语言中, **MOV** 指令用于数据传输, 段寄存器(如 **CS**、**DS**、**SS**、**ES**) 可以参与操作, 但是否能直接将段寄存器与内存单元结合使用取决于具体的操作数组合。我们来分析一下:

合法性分析

1. MOV 段寄存器, 内存单元

是合法的。例如:

asm

自动换行

复制

MOV DS, [BX]

需要Grok帮什么忙?

Grok 3

9:52

您处于离线状态

2. MOV 内存单元, 段寄存器

也是合法的。例如:

asm

自动换行

复制

MOV [BX], DS

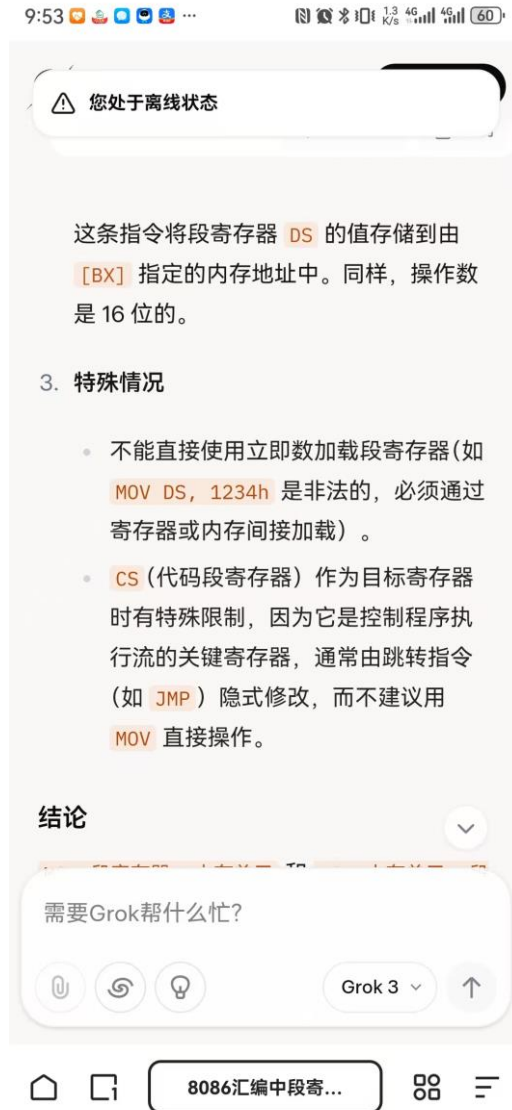
这条指令将段寄存器 **DS** 的值存储到由 **[BX]** 指定的内存地址中。同样, 操作数是 16 位的。

3. 特殊情况

- 不能直接使用立即数加载段寄存器(如 **MOV DS, 1234h** 是非法的, 必须通过寄存器或内存间接加载)。
- CS** (代码段寄存器) 作为目标寄存器

需要Grok帮什么忙?

Grok 3



如果我一开始就使用 Grok3 也许就不用走如此曲折的道路 (😄)。

【课堂补充】



知识点

1. 三大结构（冯若依曼，哈佛与改进哈佛结构）

一、冯·诺依曼结构：通用计算的基石

设计原理

冯·诺依曼结构由计算机之父冯·诺依曼于 1945 年提出，其核心特征为指令与数据共

享同一存储器，通过单一总线传输。程序执行遵循“取指→译码→执行→写回”的线性流程，所有操作按顺序完成。

优点

1. **结构简单**：硬件复杂度低，仅需单一存储器和总线，降低设计和制造成本。
2. **灵活性强**：程序与数据可动态修改，支持通用计算需求，适用于多任务场景。
3. **兼容性高**：易于扩展外设，成为早期计算机（如 ENIAC）和现代 PC 处理器（如 x86 架构）的基础。

缺点

1. **性能瓶颈**：指令与数据竞争总线资源，导致并行性受限，即“冯·诺依曼瓶颈”。
2. **安全隐患**：指令与数据混存易被恶意代码篡改（如缓冲区溢出攻击）。

应用场景

通用计算机（如个人电脑、服务器）及对成本敏感的嵌入式设备，依赖其灵活性和经济性。

二、哈佛结构：实时性的极致追求

设计原理

哈佛结构将**指令存储器与数据存储器物理分离**，采用独立总线并行访问。例如，数字信号处理器（DSP）可同时读取指令和加载数据，提升吞吐率。

优点

1. **高效并行**：指令与数据独立传输，消除总线竞争，显著加速实时数据处理。
2. **实时性强**：适用于需要确定性延迟的场景（如工业控制、音频处理）。
3. **安全性提升**：指令存储区隔离，降低代码注入风险。

缺点

1. **硬件复杂**：双存储器和总线系统增加芯片面积与功耗。
2. **灵活性不足**：程序存储器容量固定，难以动态调整代码规模。

应用场景

嵌入式系统（如 PIC 单片机）、DSP 芯片（如 TI 的 TMS320 系列）及航天器控制系统，专注于高实时性需求。

三、改进型哈佛结构：性能与灵活性的平衡

设计原理

改进型哈佛结构在物理层面保留指令与数据存储分离，但通过高速缓存（Cache）和内存管理单元（MMU）实现逻辑层面的资源共享。例如，现代 CPU 的 L1 缓存分为指令缓存和数据缓存，而 L2/L3 缓存则统一存储。

优点

1. **性能优化**：底层总线独立保证并行存取，缓存层减少内存访问延迟。
2. **动态适配**：支持程序动态加载与内存扩展，兼容复杂操作系统。
3. **功耗控制**：智能缓存管理降低高频访问能耗。

缺点

1. **设计复杂度高**：需协调多级缓存一致性，增加控制逻辑难度。
2. **成本上升**：需集成 MMU 等模块，芯片制造成本提高。

应用场景

现代高性能处理器（如 ARM Cortex-A 系列、Intel Core 系列）及移动设备 SoC，兼顾效率与通用性。

2. EU(执行单元)与 BIU

1. EU（执行单元）：计算核心

- **核心功能**：负责指令的实际运算，包括算术逻辑运算（ALU）、寄存器操作及流程控制（如条件跳转）。
- **关键特点**：
 - **专注计算**：仅处理指令译码与执行，不直接访问外部总线，依赖 BIU 提供数据。
 - **流水线优化**：通过多级流水线并行执行指令，提升吞吐率（如取指、译码、执行阶段重叠）。
 - **寄存器依赖**：运算数据主要来自寄存器（如 AX、BX），减少内存访问延迟。
- **局限性**：
 - **数据依赖瓶颈**：若 BIU 未及时提供数据，可能导致 EU 闲置，需通过乱序执行等技术缓解。
 - **功能单一性**：无法独立完成内存或 I/O 操作，必须与 BIU 协同。

2. BIU（总线接口单元）：通信枢纽

- 核心功能：管理处理器与外部总线（内存、I/O 设备）的通信，包括指令预取、数据传输及总线控制。
- 关键特点：
 - 并行预取：在 EU 执行指令时，提前从内存读取后续指令填充指令队列，减少 EU 等待。
 - 总线管理：协调地址、数据和控制总线，处理读写时序与协议（如 x86 分段机制）。
 - 缓存支持：集成缓存控制器（如 L1 Cache），加速高频数据访问。
- 局限性：
 - 带宽限制：总线速度可能成为性能瓶颈（如早期“内存墙”问题）。
 - 设计复杂度：需适配多种总线协议（如 DDR、PCIe），硬件实现复杂。

3. 协作机制与优化

1. 指令流水线：BIU 预取指令存入队列，EU 依次执行，二者并行减少空闲等待。
2. 数据供给：EU 执行中若需内存数据，向 BIU 发起请求，BIU 读取后返回结果。
3. 性能平衡：
 - EU 通过超标量、乱序执行提升计算并行度；
 - BIU 通过缓存分层、预取策略优化数据供给效率。

4. 现代架构演进

- 集成化设计：现代 CPU（如 Intel Core）将 BIU 与内存控制器集成，减少总线延迟，同时 EU 支持多核、超线程技术。
- 异构扩展：GPU、AI 加速器等专用 EU 搭配高带宽 BIU（如 HBM 显存接口），满足多样化计算需求。

总结：EU 与 BIU 体现了运算与通信分开的思想

3. 赛扬处理器

赛扬（Celeron）处理器作为英特尔面向中低端市场的产品，通过多维度技术简化与成本优化策略，在保证基础性能的同时实现显著降本。其核心手段包括：

1. 缓存精简与设计优化

赛扬系列通过大幅削减二级缓存（L2 Cache）降低芯片复杂度。早期型号甚至取消 L2 缓存，后期版本虽集成缓存至核心，但容量仅为同期奔腾系列的 1/4（如 128KB 对比 512KB），直接减少晶体管数量及制造成本。

2. 成熟技术与旧制程复用

赛扬基于已验证的旧架构（如 Pentium III 核心）及成熟制程（如 0.18 微米），避免新工艺研发的高昂投入，同时利用量产规模摊薄成本。例如，赛扬 1.2GHz 沿用 Northwood 架构，兼容旧产线设备，提升良率并缩短生产周期。

3. 功能模块与性能限制

通过移除高端指令集（如 SSE2）、禁用超线程技术、降低主频倍频设置（限制频率提升空间）等方式，减少对高性能晶体管的需求。这不仅简化芯片设计，还降低散热与封装要求，可选用低成本塑料封装方案。

4. 供应链与残次品利用

英特尔将高端芯片的残次品（如部分缓存失效的奔腾处理器）屏蔽缺陷后重定为赛扬，减少废品率并拓展利润空间。此外，通过统一封装标准与规模化采购，进一步压缩物料成本。

5. 精准市场定位

聚焦入门级用户需求，以“够用”性能换取价格优势，填补低端市场空白。此举既避免与自家高端产品竞争，又通过差异化策略巩固市场份额，形成完整产品线覆盖。

4. 高速缓存（Cache）

CPU Cache（高速缓存）是计算机体系结构中的核心组件，旨在弥合 CPU 与主存间的速度鸿沟，缓解“内存墙”问题，提升系统性能。其核心价值在于通过高速存储与智能数据管理，减少 CPU 等待数据的时间，降低总线压力，并提高数据访问效率。

1. 作用

- 速度匹配：CPU 运算速度远超内存（DRAM），Cache 以接近 CPU 频率的速度（几纳秒）提供数据，减少 CPU 空闲等待。
- 局部性利用：基于程序的时间局部性（重复访问相同数据）和空间局部性（顺序访问相邻数据），缓存高频使用的指令与数据。
- 层级优化：现代 CPU 采用多级缓存（L1/L2/L3），L1 最小最快（核心独占），L3 较大较慢（多核共享），形成速度与容量的平衡架构。

2. 工作原理

- 数据映射：内存数据按规则载入 Cache，常见策略包括直接映射（固定位置）、组相联（组内灵活）和全相联（任意位置），兼顾效率与硬件成本。
- 替换策略：Cache 满时需替换旧数据，常用 LRU（淘汰最久未用）、FIFO（先进先出）或随机替换，以最大化缓存命中率。

3. 工作流程

CPU 访问数据时，逐级查询 L1→L2→L3 Cache，命中则直接返回；若均未命中，

则从内存加载数据并按规则替换缓存块。写入时根据策略更新 Cache 或同步内存。

4. 设计优化

- 程序优化：利用局部性编写代码（如循环展开、数组顺序访问），提升缓存命中率。
- 预取机制：硬件或软件预测未来数据并提前加载，减少未命中延迟。
- 一致性协议：多核系统中通过 MESI 等协议维护多级缓存间数据一致性，避免脏数据冲突。